SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

AuraFlex Gym Management System

Group Number : 11

Name	Index Number
S.H.Chehan Mendiya	TG/IT/2025/2500
K.R.Karanayaka	TG/IT/2025/2498
R.Indumin Sasindu	TG/IT/2025/2493
T.V.K.Sendiya	TG/2024/2120
Pasindu Madusanka	TG/IT/2025/2485

1. Introduction

1.1 Purpose

This document is the Software Requirements Specification (SRS) for AuraFlex, a gym management system written in the C programming language. It explains what the system must do, so our group can use it as a guide when writing the code, testing the program, and giving the demonstration.

Many small gyms still keep their records on paper or in separate spreadsheets. AuraFlex replaces these with one console application that manages members, memberships, payments, attendance, trainers, and reports.

1.2 Scope of the System

AuraFlex handles the daily administrative work of a small gym. The main users are administrators, receptionists, and trainers. After logging in, each user sees a dashboard that matches their role.

Included in the system

- Registering, searching, updating, freezing, and unfreezing members.
- Assigning memberships to members and recording the membership payments.
- Recording and viewing payments.
- Recording check-in and check-out attendance.
- Registering trainers and managing trainer-member assignments.
- Producing member, membership, expiry, payment, and summary reports.
- Login with role-based menus, input validation, automatic ID generation where needed, and saving all records in text files inside the `data/` folder.

Not included in the system

- Online payment processing.
- A database server.
- Biometric devices such as fingerprint scanners.
- Sending emails or SMS messages.
- A graphical or web interface.
- Using the system from several computers at the same time.
- Connections to other software.
- Payroll, inventory purchasing, accounting, and medical management.

Because the records are stored in text files separated by spaces, names and descriptions must be entered without spaces. Underscores can be used instead, for example `John_Silva`.

1.3 Objectives

The main objectives of the system are listed below.

1. Allow a valid and active user to log in within a maximum of three attempts.
2. Show dashboard options according to the role of the logged-in user.
3. Prevent duplicate member, trainer, membership, and assignment IDs.
4. Save every valid record in the correct file inside the `data/` folder.

5. Prevent payments for members who do not exist or are not active, and reject amounts that are zero or negative.
6. Prevent a member from having more than one open attendance record on the same day.
7. Produce reports using the data currently stored in the data files.
8. Reject wrong menu input, invalid dates, unsupported payment methods, and invalid references to other records, and show a clear message to the user.

1.4 Intended Users

- **Administrator:** can use all member, membership, trainer, payment, attendance, and report functions.
- **Receptionist:** can register and view members, manage memberships, record payments, and handle attendance.
- **Trainer:** can view the members assigned to him or her, and add or remove assignments.
- **System maintainer / developer:** looks after the source code, data file layout, compiling, and testing. This is not an end-user role.

1.5 Development Environment and Constraints

Area	Details
Programming language	C, using procedural and modular programming
Operating environment	Terminal / command-line environment. Password masking uses `conio.h`, so it may not work the same way on every platform.
User interface	Menu-driven console interface with formatted tables, prompts, and status messages
Data storage	Plain text files in the `data/` folder, with fields separated by spaces. Temporary files are used when records are rewritten.
Expected concepts	Structures, functions, header and source files, file handling, string handling, input validation, date and time functions, role-based control flow, and replacing files using temporary files

The program must be run from the project folder so that the relative `data/` paths work. It does not lock files when writing, and passwords and records are not encrypted because they are saved as plain text.

2. Overall System Description

2.1 Background / Problem Description

Gym staff have to look after member registration, membership periods, fee collection, attendance, trainer assignments, and reports. When this is done with paper records or separate spreadsheets, several problems can happen:

- The same ID may be given to more than one person.
- Payment history can be lost.
- Attendance records can be wrong, for example a member who has already left still looks like he is inside.

- Membership expiry dates can be missed.
- It is hard to limit what each staff member is allowed to do.

2.2 Proposed Solution

AuraFlex puts all of these activities into one C application. The user first logs in and then uses a dashboard made for his or her role. Each component checks the input and the related records before saving any change. The records are kept in text files, so no database server is needed, and the report functions read the same files to show up-to-date results.

2.3 Main System Components

The system has six integrated components.

1. **Authentication and role access:** reads the users file, checks the username, password, and active status, limits the login attempts, and selects the correct dashboard.
2. **Member and membership management:** manages member details and status, and assigns memberships with the plan, dates, and fee.
3. **Payment management:** records payments, supports membership payments, searches the payment history, and calculates totals.
4. **Attendance management:** records check-in and check-out, shows attendance history, and gives a daily attendance summary.
5. **Trainer management:** manages trainer records and the relationships between trainers and members.
6. **Reports and shared user interface:** shows the menus and produces the member, membership, expiry, payment, and summary reports from the shared data.

2.4 Overall System Workflow

The diagram below shows how a user moves through the major components of the system.



Figure 1: Overall workflow of AuraFlex

3. Functional Requirements

The functional requirements are given separately for each component.

3.1 Authentication and Role Access

ID	Requirement
FR-AUTH-01	The system shall read the user ID, username, password, role, and status from `users.txt`.
FR-AUTH-02	The system shall allow login only when the username and password match and the user status is `ACTIVE`.
FR-AUTH-03	The system shall hide the password while it is typed (where supported) and allow no more than three attempts each time the program is started.
FR-AUTH-04	The system shall show the administrator, receptionist, or trainer dashboard according to the role of the logged-in user.
FR-AUTH-05	The system shall allow the user to log out and end the current session.

3.2 Member and Membership Management

ID	Requirement
FR-MEM-01	The system shall register a member with a unique member ID, name, phone number, email, and initial status.
FR-MEM-02	The system shall display all members and search for a member by member ID.
FR-MEM-03	The system shall update member details without damaging the other records.
FR-MEM-04	The system shall freeze and unfreeze a member by changing the stored status.
FR-MEM-05	The system shall assign a membership only to an existing member and store the plan, dates, fee, status, and a unique membership ID.
FR-MEM-06	The system shall display membership records and identify active memberships that have expired, using the current date.
FR-MEM-07	Assigning a membership shall call the membership payment process. The assignment shall not be shown as successful if the payment fails.

3.3 Payment Management

ID	Requirement
FR-PAY-01	The system shall record a payment for an existing active member.
FR-PAY-02	The system shall generate a unique payment ID by checking the existing payment records.
FR-PAY-03	The system shall store the member ID, a positive amount, the payment date, and the payment method.
FR-PAY-04	The system shall accept only the payment methods `CASH`, `CARD`, and `ONLINE`.
FR-PAY-05	The system shall search the payment history of a member, display all payments, and calculate the total amount recorded.
FR-PAY-06	The system shall provide a payment function that the membership assignment can use.

3.4 Attendance Management

ID	Requirement
FR-ATT-01	The system shall record an attendance ID, member ID, date, check-in time, and check-out time.
FR-ATT-02	Check-in shall be allowed only for an existing active member.
FR-ATT-03	The system shall reject a second open check-in for the same member on the same day.
FR-ATT-04	The system shall check out the open record of the member and keep all other attendance records unchanged.
FR-ATT-05	The system shall search attendance by member, display the attendance history, and give a daily count of check-ins, completed check-outs, and members still inside.

3.5 Trainer Management

ID	Requirement
FR-TRN-01	The system shall register trainers with unique IDs and their professional details.
FR-TRN-02	The system shall view, search, update, activate, and deactivate trainer records.
FR-TRN-03	The system shall assign a member to a trainer only when both exist and the trainer is active.
FR-TRN-04	The system shall reject a duplicate trainer-member assignment.
FR-TRN-05	The system shall remove and view trainer-member assignments.

3.6 Reports and User Interface

ID	Requirement
FR-REP-01	The system shall provide member, membership, expired membership, payment, and summary reports.
FR-REP-02	Reports shall read the records saved in the data files, not only the data in memory during the current session.
FR-REP-03	The interface shall show clear menus, prompts, success messages, validation messages, and file error messages.
FR-REP-04	After an operation, the system shall let the user return to the dashboard or log out.

4. Data Requirements

All the files below are persistent files stored in the `data/` folder. Each record is saved as one line with fields separated by spaces.

File	Purpose and main data
`data/users.txt`	Login details, role, and active / inactive status of users
`data/members.txt`	Member ID, name, phone, email, and member status
`data/membership_plans.txt`	Available membership plans and their fees
`data/memberships.txt`	Membership ID, member ID, plan, start and end dates, fee, and status
`data/payments.txt`	Payment ID, member ID, amount, date, and payment method
`data/attendance.txt`	Attendance ID, member ID, date, check-in time, and check-out time

File	Purpose and main data
data/trainers.txt	Trainer details such as contact, specialization, schedule, and status
data/trainer_members.txt	Trainer ID and member ID pairs for assignments
data/exercises.txt	Exercises used for workout plans and reference
data/workout_plans.txt	Saved workout plan records
data/diet_plans.txt	Saved diet plan records
data/goals.txt	Fitness goals of members and their targets
data/progress.txt	Progress measurements and updates of members
data/classes.txt	Gym classes and their schedules
data/bookings.txt	Member bookings for classes or services
data/equipment.txt	Equipment records
data/maintenance.txt	Equipment maintenance records and status
data/notifications.txt	Notification messages for the system or members
data/audit_log.txt	History of system events, where logging is enabled

The file `data/temp.txt` and other temporary files are only working files used while records are updated or deleted. The original file is replaced only after the new file has been written successfully. Temporary files are not business records and are not used for reports.

5. System Functions Summary

Component	Important functions
Authentication	`login`, role dashboard selection, logout
Members	`memberMenu`, `registerMember`, `viewMembers`, `searchMember`, `updateMember`, `freezeMember`, `unfreezeMember`
Memberships	`membershipMenu`, `assignMembership`, `viewMemberships`, `recordPayment`, `viewPayments`
Payments	`paymentMenu`, `paymentRecord`, `processMembershipPayment`, `searchPaymentHistory`, `viewPaymentHistory`, `calculateTotalPayments`
Attendance	`attendanceMenu`, `recordCheckIn`, `recordCheckOut`, `searchAttendance`, `viewAttendanceHistory`, `dailyAttendanceSummary`
Trainers	`trainerMenu`, `registerTrainer`, `viewTrainers`, `searchTrainer`, `updateTrainer`, `changeTrainerStatus`, `assignMemberToTrainer`, `removeMemberFromTrainer`, `viewTrainerMembers`
Reports	`reportsMenu`, `memberReport`, `membershipReport`, `expiredMembershipReport`, `paymentReport`, `summaryReport`
Interface / system	`printLogo`, `displayDateTime`, `getDateTime`, `clearScreen`, `clearScreen2s`

6. Input Validation

6.1 Component-Level Validation

- **Authentication:** username and password must be entered, they must match a record, the user must be active, and the number of attempts is limited.
- **Members:** fields cannot be empty, the ID must have a valid format, duplicate IDs are not allowed, and only supported status changes are accepted.
- **Memberships:** the member must exist, the membership ID must be unique, the date range and plan must be valid, and the fee must be a valid amount.
- **Payments:** the member must exist and be active, the amount must be a positive number, the date must be a valid calendar date, and the method must be `CASH`, `CARD`, or `ONLINE`.
- **Attendance:** the member must exist and be active, the member must not already have an open record that day, and the check-out must match an open record.
- **Trainers:** the trainer ID must be unique, details and status must be valid, the trainer must be active and the member must exist for an assignment, and duplicate assignments are not allowed.
- **Menus:** choices must be numbers within the menu range, and non-numeric input must not crash the program.

6.2 Cross-Component Validation

- A membership, payment, attendance entry, or trainer assignment must refer to an existing member.
- Payments and check-ins must be rejected for frozen or inactive members.
- A membership payment must belong to the member and the amount of the membership being assigned.
- A trainer assignment must refer to an existing active trainer and an existing member.
- Reports must handle missing or empty data files and must not show a failed file read as a successful report.
- When a new ID is generated, the existing records must be checked first to avoid duplicates.
- Updates and deletions must keep all records that do not match the selected ID.

7. Non-Functional Requirements

7.1 Usability

- Menus use consistent numbering and clear option names.
- Prompts and error messages explain what input is expected or why an operation was rejected.
- The user is returned to the correct menu after each operation.
- Each role dashboard shows only the operations meant for that role.

7.2 Reliability

- Valid records remain available after the program is closed and started again.
- When records are rewritten, a temporary file is used and then replaces the original file.

- A missing, unreadable, or unwritable file gives a clear error and never a false success message.
- Unrelated records are kept safe during updates and deletions.

7.3 Data Persistence

- Every accepted record is written to its own file.
- Dates, times, IDs, statuses, and amounts are stored in a format that can be read again in later sessions.
- Reports take their results from the saved files.
- Plain-text storage is a project limitation. Passwords and records are not encrypted.

7.4 Maintainability

- Public functions are declared in header files and written in separate source files for each module.
- Shared structures and validation rules are reused instead of being copied.
- File paths and record formats stay the same across all modules.
- New components can be added by using clear functions and header files.

7.5 Performance

- Normal menu operations should respond quickly for the small amount of data in a gym.
- Searches, totals, and reports may read the text files from start to end.
- ID generation and reports should avoid reading the same file many times when one read is enough.

7.6 Error Handling

- Invalid input is rejected with a clear message, and the user can try again.
- Errors when opening, reading, writing, renaming, or updating files are reported to the user.
- If an operation that uses another component fails, no success message is shown.
- If the login fails after the allowed attempts, the program shows an access denied message and ends.

8. System Integration

The file `main.c` is the entry point of the application. It calls the `login` function, receives the record of the logged-in user, and sends the user to the dashboard for his or her role. The dashboard then calls the main menu function of each component, which are declared in the component header files.

The components work together by sharing the following operations and data.

- Membership management calls `processMembershipPayment` in payment management, so assigning a paid membership also creates a payment record.
- Payment and attendance management read `members.txt` to check that the member exists and is active.
- Trainer management reads the member and trainer files before adding an entry to `trainer_members.txt`.

- Reports read the member, membership, payment, and attendance data to calculate the current results.
- The shared interface and system functions give the same display, date and time, screen clearing, and pause behavior everywhere.

In short, the flow of the system is: login, role control, component operation, validation using the shared records, saving to files, reports, and then back to the dashboard or logout.

9. Appendices

Appendix A: Source Layout

Folder	Contents
`include/`	Header files and shared declarations
`src/`	C source files for the modules, including `main.c`
`data/`	Text files that store the records, and temporary working files
`build/`	Compiled programs and test outputs
`docs/`	Project reports and requirement documents

Appendix B: Role Access Summary

Operation	Administrator	Receptionist	Trainer
Member management	Yes	Limited	No
Membership management	Yes	Yes	No
Payment management	Yes	Record payment only	No
Attendance management	Yes	Yes	No
Trainer management	Yes	No	Assignment operations only
Reports	Yes	No	No

Appendix C: Key Business Rules

1. Only active and authenticated users can enter the system.
2. A frozen member cannot make a payment or check in.
3. A member cannot have two open check-ins on the same day.
4. A trainer-member assignment can exist only once.
5. Expiry reports compare the membership end dates with the current date.
6. If a file operation or a related-record check fails, a success message must not be shown.